Development and validation of ADAS perception application in ROS environment integrated with CARLA simulator

Stevan Stević, Momčilo Krunić, Marko Dragojević and Nives Kaprocki, *Members, IEEE*

***Abstract* — Advancing to higher levels of driving automation brings unpredicted challenges and with them, many situations that cannot be foreseen. In order to overcome these problems, set of functionalities in modern vehicle is growing in terms of algorithmic complexity and required hardware. Risk of testing implemented solutions in real world is high, expensive and time consuming. That is why virtual simulation tools for automotive testing are heavily acclaimed. Original Equipment Manufacturers (OEMs) use these tools to create closed sense, compute, act loop. Production software is tested against simulated sensing data and simulated action consequences are generated according to the given software commands. This gives OEMs ability to optimize design of the vehicles before any physical prototypes are produced. Early optimization brings reduced costs and less time delays. This paper presents development of simple C++ perception applications using ROS as a prototyping platform that are validated and tested with "Software-In-the-Loop" (SIL) methods. Simulations are created using CARLA simulator to provide data and transform commands given by the autonomous platform into simulated actions. Validation is done by connecting Autoware autonomous platform with CARLA simulator in order to test against various scenes in which applications are applicable.**

***Keywords* — Autonomous driving, perception, ROS, CARLA, AUTOWARE, SIL, ADAS, C++, Python.**

I. Introduction

Development of autonomous vehicles is a major trend in automotive industry. Pushing towards SAE levels [1] four and five and fully automated vehicle as an ultimate product, engineers are facing issues that have never been addressed. As they progress with development of some functionality new problems arise because of uncertainty of physical world, as there are many unpredicted situations that could cause accidents. Due to this, development of virtual simulators to test vehicle's cognitive computing [2] becomes crucial part of the development. With this approach the perception module [3] receives input from computer-generated scenes and mathematically modelled movement patterns for pedestrians, bicycles, and other entities. Acting module [3] on the other side outputs commands to simulators that implement these as actions. Using this, billions of kilometres that are required [4] to demonstrate reliability of autonomous vehicles in terms of fatalities and injuries, have been already simulated by OEMs [5]. By using simulator's abstract visualizations, engineers can focus on development of core capabilities for autonomous driving, such as: driving models and systems, remote assistance, mapping, localization, perception etc.

This paper emphasizes and explains the importance of using simulators in the modern automotive development and gives practical example by showing how simple advanced driver-assistance system **(**ADAS) applications can be tested within the context. The rest of the material is organized as follows. First part explains current state of the industry, problems and practices used. Also, provides some academical and industry related as a background. Second section explains platforms and tools that are for development and simulations. Section IV describes the purpose of test applications and presents existing setup for connecting Autoware [6] with CARLA and describes sensor and start up file configuration. Section V presents validation for these use-cases and final section concludes the paper with the review on work done and some future steps.

II. Simulators

Simulators use different models of environments that can be built from high resolution LiDARs, cameras or even virtual maps that provide annotations with tools like OpenDRIVE [7]. Furthermore, some types of simulators can augment existing data like point cloud to create obstacles [8]. Based on this, modern day simulators like rFpro [9], LGSVL [10], AVS [11] by Uber, CARLA [12], DRIVE CONSTELLATION Simulator [13] by NVIDIA and others are able to create a very realist scenes and complex layouts like road paint or road separation that can be difficult to discern even for humans. On top of that, these ecosystems are scalable and can implement different feature requests, unlike early development tools of this type that stretch their capabilities to satisfy different use-cases.

Simulators usually offer simulation of wide variety of

Stevan Stević is with the RT-RK Institute for Computer Based Systems, Novi Sad, Serbia (e-mail: stevan.stevic@rt-rk.com).

Momčilo Krunić is with the Faculty of Technical Sciences, University of Novi Sad, Serbia, Novi Sad, Serbia (e-mail: momcilo.krunic@rt-rk.com).

Marko Dragojević is with the RT-RK Institute for Computer Based Systems, Novi Sad, Serbia (e-mail: marko.dragojevic@rt-rk.com).

Nives Kaprocki is with the RT-RK Institute for Computer Based Systems, Novi Sad and Faculty of Technical Sciences, University of Novi Sad, Serbia (e-mail: @rt-rk.com).

978-1-7281-4790-1/19/$31.00 ©2019 IEEE

virtual sensors that can be placed on the ego vehicle to recreate real test vehicles. These sensors can provide stream of different formats of data like LiDAR point clouds [14], RGB, HDR, depth in meters or material properties of objects and others. The data is produced based on current scene in the simulator and virtual placement of the sensors.

Problem with using synthesized and generated data comes from stochastic nature of the real world by which it avoids patterns found in these types of data and can notably impair performance of trained neural networks or prediction models. Simulators tend to overcome these problems by allowing stochastic behaviours to be injected with disruptive, low likelihood-of-occurrence events. This is utilized for example, by creating unpredictable behaviours for drivers, on some random number of driven kilometres.

The ability to have a simulation that correlates closely with physical world, with different scenes, traffic flow and virtual sensors, provides engineers environment to create SIL [15] systems. This means that every entity like hardware, data, use-cases and such are simulated, whilst using production software to test against it. By testing some features earlier in the design cycle, that would otherwise require finished prototypes, changes are less expansive to implement, and problems are eliminated or predicted on time. Except these advantages, SIL testing also provides a level of certainty, because engineers can be sure that given behaviour is not due to any mechanical or electrical malfunctions but caused by a written software.

It is hard to say which simulators are "state-of-the-are" products, because these metrics can only be defined in the scope of requirements that are being tested. Raging from simulators for algorithm testing to sensor technology readiness level (TRL). However, latest simulators are being designed to work with client-server architecture which is also a case with CARLA simulator. This approach generated a new set of scenarios were multiple ego vehicles can be used in the same scenes. Furthermore, even bigger advantage is that simulator can be cloud-based, simplifying configurations, organizations and even providing opportunities for research to departments that are lacking the technical or finical resources.

III. Platforms and Tools

In order to fulfil the goal of using applications in SIL environment we needed fast prototyping tools, autonomous platform to build upon and simulation tool. For prototyping we used Robot Operating System (ROS) [16]. ROS represents a software stack with set of tools that can be used for robot control and analysis and for that used to put together various robotics solutions. It uses concept of nodes (processes) that communicate between themselves using topics and services. This is orchestrated by one main node called Master. ROS also offers multilingual bindings making it easy to integrate with any existing code base. For these reasons many researchers and even companies use ROS as base for autonomous agents. This is also a case with Autoware platform which provides functional components required to accomplish autonomous driving. Platform architecture and set of algorithms in Autoware is well suited for urban driving scenarios. Also, for processing purposes the platform utilizes GPU optimized frameworks and libraries like OpenGL, CUDA, OpenCL, PCL etc.

Large code base, user community haven't been neglected by CARLA developers, so they created ROS Bridge [17] component for CARLA simulator. As the simulator itself is based on Unreal Engine 4 [18] and Python API there was a need to transform data into ROS format of messages for self-driving software to run on CARLA as it is, without modifications. This provided ground to connect CARLA with Autoware, which is done with the "Autoware in Carla" [19] integration stack that relies on existing bridge. The bridge contains three Carla clients:

1. ROS Bridge - Monitors existing actors in Carla, publishes changes on ROS Topics (e.g. new sensor data)
2. Ego vehicle - Instantiation of the ego vehicle with its sensor setup.
3. Waypoint calculation - Uses the Carla Python API to calculate a route.

With these CARLA clients, overall architecture is presented in Figure 2.

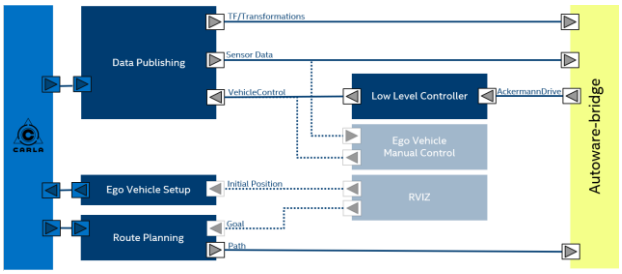

Figure 1. Architecture of CARLA and Autoware integration

For visualization of actions and information we used RViz tool from the ROS environment.

IV. System Integration

In this section, the development of demo applications and their integration with Autoware have been provided. Furthermore, sensor configuration for ego vehicle in CARLA simulator and configuration of start-up scripts that create virtual environment alongside virtual vehicle is described. Some of the parameters given are weather conditions, number of vehicles and pedestrians, selection of maps with waypoints etc.

A. Demo ADAS applications

First part of the integration was to implement two demo ADAS applications that could be used in simulation testing. One application was based on implementation of occupancy grid framework [20] which is mostly used for mapping use-cases. The other one is implementation of stopping distance monitor functionality which can be used in system like Forward Collision Warning (FCW). Both applications were implemented in C++ for performance

reasons and aspects of integration with modern automotive platforms.

Both applications were designed to be used with LiDAR data, more precisely point cloud input streams. Autoware as platform for an autonomous driving already has nodes that can publish point cloud data on defined topics. Because of this, applications were implemented as ROS packages within the Autoware platform.

Based on input data, we create parameterized occupancy grid that can take size of the grid [21], topic for point cloud data, and size of every cell with ego vehicle in the centre of the grid. After, the data is received transformation is done from LiDAR coordinate system to vehicle perspective. Ego vehicle usually has a fixed frame for centre of its own coordinate system that is used to align all algorithms, sensors and maps. This is also given as a parameter for package and rear axis is used in this demonstration. Next is filtering of Region of Interest (RoI) based on given size to reduce cost of updating cells. Parallel with this, ground points are being filtered, which is done to remove false detections and to provide list of these points that could be use in detection of free space. The update of the cells is the end of cycle processing which is done according to log-odds notation. This workflow can be seen in Figure 3.

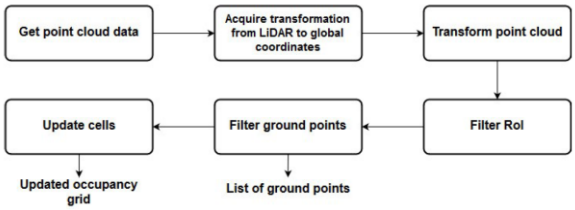

Figure 2. Occupancy grid package processing

Implementation of the second application, the stopping distance monitor [22], required additionally velocity of the vehicle. This is needed in order to calculate if the ego vehicle could stop before it hits the vehicle in front. In this example LiDAR data is used to determine distance between the vehicles. To acquire this data, application must subscribe to topics that are advertised by corresponding nodes. Because of this application implements communication adapter that handles communication with ROS stack. Using this design, computational logic can be independent of ROS and can be used in other future stacks. Communication module forwards data to the computational module. Point cloud data is clustered into groups and by determining nearest detected cluster, which is in the vehicle path, distance between two vehicles is determined. This information, together with the current speed and maximum deceleration model can generate warning if vehicle would not be able to top on time. This is done by generation module which forwards the warning to communication module that is also implemented to publish back into the ROS environment. This information can be used by different features to execute automatic braking or notify the driver. Design of the application can be seen in Figure 4.

In this stage of development, pre-recorded *rosbag* files have been used from real autonomous test drives. With this approach we could only obtain the data and test if there are visible markers in RViz visualization tool. Problem with this approach is that it does not provide a closed feedback loop, because it can't execute actions based on given commands. For this reason, CARLA simulator has been introduced.

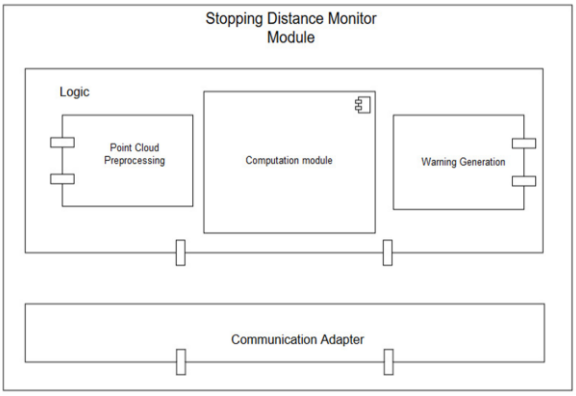

Figure 3. Stopping distance monitor application design

B. Sensors

The goal of integration with Carla is to get data from the simulator to the Autoware. This is done by utilizing previously described setup with ROS Bridge for Carla messages and connection with Autoware. In order to send the simulated data first step is to create it with sensors from Carla database. As both applications needed point cloud data, *lidar.ray_cast* sensor is instantiated on top of the vehicle. In addition to this *camera.rgb* sensor is added to visualize the scene in RViz. Besides these that are directly important for the application, other sensors like GNSS and HD map provide are instantiated. Autoware nodes responsible for localization utilize this type of sensors. All sensors are described in JSON file that is consumed by simulator.

C. Scene configuration

Final step needed to run the whole SIL setup, was to create virtual scenes. Carla already offers Python API to create diverse scenes with existing maps which we utilized to get optimal setup for our use-cases.

V. Validation

Once the custom scene was created, all the nodes and bridges started along with the vehicle model loaded in the environment with instantiated sensors. The vehicle was controlled with RViz by setting a destination point to which it moved by existing waypoints on the map.

As depicted in the Figure 5, the vehicle model and the custom scene was created with Non-Player Characters (NPC) – vehicles. These vehicles spawned deliberately in front of the ego vehicle to test if the vehicle will stop. Vehicle was aware of obstacles in front which is visualized with the red part of the rectangle and has generated the warning signals and executed stoppage manoeuvre. Similar use cases were created for mapping application where we wanted to have as much objects as possible to test our

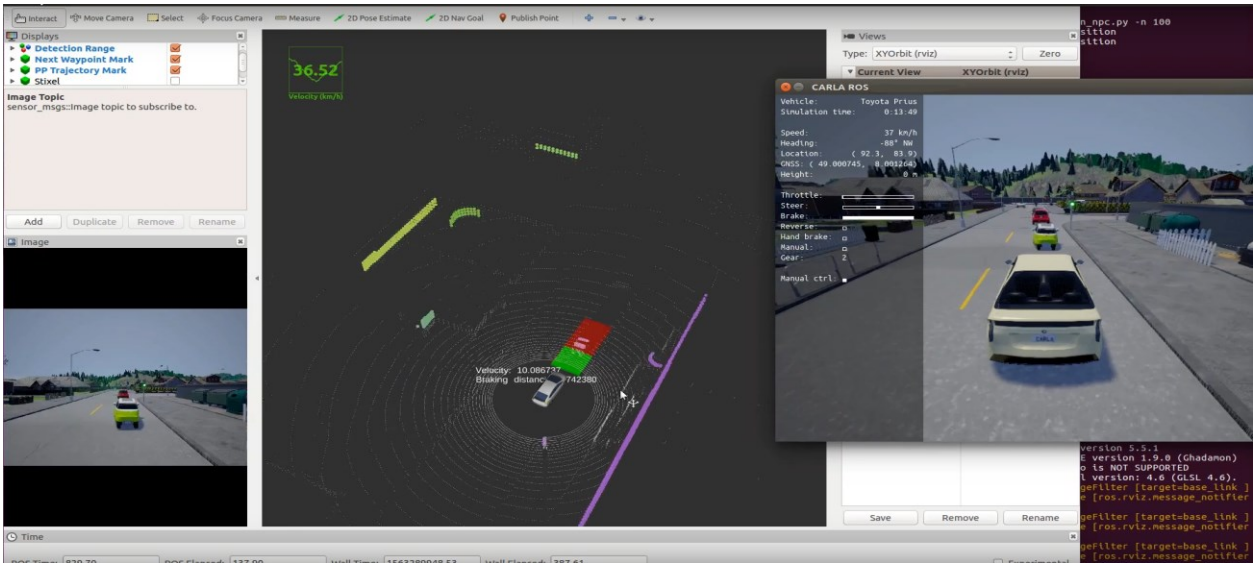

Figure 4. Visualization of FCW application integrated with CARLA simulator

application with different cell size and size of the affected frame

The camera images (bottom left), joint motions and the LiDAR scans that generated from Carla were visualized in RViz, but this could also be done with depth sensors and many others.

Except visual confirmation of correctness, other formal methods were used during the testing as explained in [21] and [22]. Applications were written by Test Driven Development methodology. This ensured testing on both unit and acceptance level of testing.

VI. Conclusion

This paper emphasized importance and benefits of early software testing and presented practical example of simple SIL setup. Setup consisting of Autoware and Carla allowed us to have complete feedback loop for applications mentioned in [21] and [22]. This setup enabled us for future research testing and better understanding of the whole development process. ROS and Autoware were chosen as prototyping platforms as they have similar communication mechanisms and principles as modern automotive platforms (e.g. Adaptive ATUSOAR). This can affect design of the application from start and provide easy transition to such platforms with least costs. CARLA simulator is open source software and supports integration with Autoware and rich Python API, which is why we used it before other simulators.

Acknowledgment

This work was partially supported by the Ministry of Education, Science and Technological Development of Republic of Serbia under Grant III_044009_1

References

[1] "SAE levels of driving automation", https://www.sae.org/news/2019/01/sae-updates-j3016-automated-driving-graphic [Accessed: September 2019].

[2] S. Behere and M. Torngren, „A functional architecture for autonomous driving", 2015 First International Workshop on Automotive Software Architecture (WASA), Montreal, QC, 2015, pp. 3-10.

[3] Matthaei, Richard & Maurer, Markus, „Autonomous driving – A top-down-approach", Automatisierungstechnik, 63, 2015.

[4] Kalra, Nidhi and Susan M. Paddock, Driving to Safety: How Many Miles of Driving Would It Take to Demonstrate Autonomous Vehicle Reliability? Santa Monica, CA: RAND Corporation, 2016.

[5] "Waymo simulated kilometres", https://techcrunch.com/2019/07/10/waymo-has-now-driven-10-billion-autonomous-miles-in-simulation/ [Accessed: September 2019].

[6] "Autoware.io", https://gitlab.com/autowarefoundation/ arla re.ai [Accessed: September 2019].

[7] "OpenDRIVE", http://www.opendrive.org/ [Accessed: September 2019].

[8] Fang, J., Zhou, D., Yan, F., Zhao, T., Zhang, F., Ma, Y., Wang, L., & Yang, R. (2018). Augmented LiDAR Simulator for Autonomous Driving.

[9] "rFpro", http://www.rfpro.com/ [Accessed: September 2019].

[10] "LGSVL Simulator", https://www.lgsvlsimulator.com/ [Accessed: September 2019].

[11] "AVS", https://avs.auto/ [Accessed: September 2019].

[12] Dosovitskiy, A., Ros, G., Codevilla, F., López, A., & Koltun, V. (2017). CARLA: An Open Urban Driving Simulator. CoRL.

[13] "NVIDIA DRIVE CONSTELLATION", https://www.nvidia.com/en-gb/self-driving-cars/drive-constellation/ [Accessed: September 2019].

[14] Fang, Jin & Yan, Feilong & Zhao, Tongtong & Zhang, Feihu & Zhou, Dingfu & Yang, Ruigang & Ma, Yu & Wang, Liang. (2018). Simulating LIDAR Point Cloud for Autonomous Driving using Real-world Scenes and Traffic Flows.

[15] Sooyong Jeong, Yongsub Kwak and Woo Jin Lee, "Software-in-the-Loop simulation for early-stage testing of AUTOSAR software component," 2016 Eighth International Conference on Ubiquitous and Future Networks (ICUFN), Vienna, 2016, pp. 59-63.

[16] "Robot Operating System", https://www.ros.org/ [Accessed: September 2019].

[17] "ROS bridge for CARLA simulator", https://github.com/carla-simulator/ros-bridge [Accessed: September 2019].

[18] "Unreal Engine 4", https://docs.unrealengine.com/en-US/index.html [Accessed: September 2019].

[19] "Autoware in Carla", https://github.com/carla-simulator/carla-autoware [Accessed: September 2019].

[20] A. Elfes, "Using occupancy grids for mobile robot perception and navigation," in *Computer*, vol. 22, no. 6, pp. 46-57, June 1989.

[21] Stevan Stevic, Marko Krnjetin, Nenad Cetic and Nives Kaprocki, „Parameterized Occupancy Grid as a Base for Perception Applications in ROS Environment", IcETRAN 2019.

[22] Marko Dragojevic, Momcilo Krunic, Ninoslav Jovanov and Nemanja Lukic, „ROS as a Rapid Prototyping Platform for LIDAR Based Stopping Distance Monitor", IcETRAN 2019.